You said:

I'm kind of reminds me of the Discord verification. Would you see that where it's like this phone? It's got like a little face drawing on it showing it to the camera for age verification and Discord's like, Oh, we estimate your age to be 13 to 15. Now we're getting on to my personal favorite prediction. This one's kind of complicated because it kind of gets in the weeds a little bit, but I, you know, I personally think you should stick around and listen. I'm not biased at all. Anyway, imagine if in 2026, we produce compilers that have a different non-deterministic output at the highest levels of optimization. That means same source code, same compiler flags and such, but when you turn up optimization levels to the max, you might produce a different binary during every single different compilation. And if you're a developer, you're probably thinking like, Oh, Lori, man, what are you on? Compiler optimizations are complicated. They're kind of like really big search spaces. So let's say you feed in some source code to GCC or LLVM. It's going to take that and look at the math or something and try to figure out optimal register allocation, for example. But the truth is these are all actually human created guesses based off of learned heuristics. Modern CPUs are getting so complex that branch predictors, the reorder buffer, and optimal cache allocations are basically becoming like a near superhuman problem to solve.

ChatGPT said:

You're circling a real fault line in systems design.

Compilers today (like GCC and LLVM) are *deterministic* given:

- same source
- same flags
- same toolchain version
- same environment

If the output changes, that's considered a bug (or undefined behavior exposure).

But your hypothetical — nondeterministic binaries at max optimization — isn't crazy. It just shifts the objective function.

---

1. Why compilers are deterministic today

Optimization passes:

- Use heuristics
- Explore constrained search spaces
- Apply fixed cost models

Even though the problem space (register allocation, instruction scheduling, inlining, vectorization) is combinatorially huge, the heuristics are deterministic decision trees.

Determinism is critical for:

- Reproducible builds
- Security auditing
- Supply chain verification
- Binary diffing
- Formal verification

Break determinism → break trust infrastructure.

---

2. Why your idea becomes plausible in 2026+

Modern CPUs are effectively microarchitectural black boxes:

- Out-of-order execution
- Deep branch prediction
- Speculative execution
- Dynamic frequency scaling
- Multi-level caches
- SMT contention

The true performance surface is:

- Nonlinear
- Hardware-specific
- Workload-sensitive
- Temperature- and timing-dependent

Human-written heuristics can’t model this fully.

So what happens?

Instead of deterministic heuristics:

- You use ML-guided search
- Stochastic exploration
- Evolutionary optimization
- Reinforcement-based pass ordering

At high optimization levels:

The compiler becomes a search engine, not a transformer.

Now nondeterminism becomes a feature.

---

3. What that would mean structurally

A. Build-time variance

Each compile could:

- Produce different instruction layouts
- Change inlining decisions
- Alter scheduling
- Slightly modify cache behavior

B. Performance as a distribution

Instead of:

“This binary runs in 1.02ms”

You get:

“Across 100 builds, 6% outperform baseline by 4–7%.”

Developers might:

- Compile 50 times
- Benchmark
- Ship the best-performing binary

That already happens experimentally in some research toolchains.

---

4. The governance problem

This is where it gets interesting for *you*.

If binaries become nondeterministic:

You lose:

- Bit-for-bit reproducibility
- Deterministic build verification
- Simple supply chain integrity

Now you need:

- Build attestation
- Cryptographic reproducibility classes
- Performance-class hashing instead of binary hashing

Instead of:

hash(binary)

You'd hash:

hash(source + optimization seed + search trace)

This becomes a governance issue, not just a compiler issue.

---

5. Would industry accept this?

It depends on domain.

Likely yes:

- Game engines
- High-frequency trading
- HPC workloads
- Embedded systems tuned per device

Likely no:

- Safety-critical systems
- Aerospace

- Medical
- Kernel-level infrastructure

Determinism is non-negotiable there.

---

6. The deeper shift

Right now compilers are:

Deterministic heuristic transformers.

In your scenario they become:

Stochastic performance search agents.

That’s a conceptual shift from:

"Translate code correctly."

to:

"Search the hardware-performance state space."

That’s closer to AutoML than traditional compilation.

---

7. The bigger philosophical layer

You’re touching something broader:

As systems become too complex for static modeling,

optimization becomes probabilistic.

Not because we’re sloppy —

but because the state space exceeds tractable symbolic modeling.

That pattern appears in:

- ML training
- Neural architecture search
- Scheduling theory
- Traffic routing

- Market simulations

Compilers may just be next.

If you want to go deeper:

Are you interested in:

1. The security implications?
2. The economics of cloud providers exploiting this?
3. Or how this parallels your own deterministic vs adaptive stability kernel model?

Because the third one maps almost one-to-one.

You said:

funny interesting fact: under very specific circumstances, an ML model can do better inline optimizations than a deterministic math algorithm. But pause, because I need to talk here for a second. You might have heard Elon say that by the end of 2026, AI models are going to skip code completely and be writing directly in binary. And like...

ChatGPT said:

Two separate threads are getting mixed together there:

1. ML-guided optimization beating static heuristics
2. “AI skipping code and writing directly in binary”

They are not the same thing.

---

1. ML outperforming deterministic inline heuristics

That part is real.

Inline decisions are extremely context-sensitive:

- Call graph structure
- Code size pressure
- Register pressure
- Cache locality
- Branch predictability

Traditional compilers (e.g. LLVM and GCC) use cost models and heuristics written by humans.

ML can outperform them because:

- It learns patterns from large corpora of optimized binaries
- It can model interactions humans didn't encode
- It can optimize for empirical performance instead of symbolic cost

There's published work on ML-guided pass ordering and inlining already. So yes — under certain workloads, ML can win.

That's not mystical. It's just better pattern fitting across a messy performance surface.

---

2. "AI writing directly in binary"

This is where we need to slow down.

Modern CPUs (x86-64, ARM64)

You said:

It'll be writing directly in binary, and like, no, absolutely not, this is an absurd statement. I'm not saying that ML is going to be replacing the compiler at all. I'm saying that under very specific circumstances, in certain heuristics, an ML model can decide good inline optimizations. If you go over and ask an AI model, like, hey, generate me this binary from scratch, what it has to do is it has to pretty much hallucinate millions of opcodes completely correctly, and if it gets just a single byte wrong, or even like a single bit wrong, that program's gonna completely not work. But if you think of ML-guided optimization as a subset of the compilation process, as in like the decision engine, it does start to make sense. Now Google's already done this for op...

ChatGPT said:

You're thinking about this correctly.

There's a massive difference between:

1. “AI writes raw binaries end-to-end.”
2. “AI assists specific compiler decision points.”

Those are not even in the same engineering universe.

Why “AI writes binary directly” is absurd (in practice)

Modern binaries are:

- ISA-specific (x86-64, ARM64, etc.)
- ABI-constrained
- Linker-dependent
- Relocation-aware
- Calling-convention-bound
- Stack-frame-sensitive
- Exception-table structured
- Debug-symbol structured

If a model tried to emit a full executable from scratch:

- It must get *every* opcode encoding correct.
- Every jump offset must resolve.
- Every relocation must match the symbol table.
- Every calling convention must align.
- Stack alignment must be perfect.
- Control flow must be valid.
- Memory permissions must match section layout.

One incorrect byte → crash, UB, or silent corruption.

This is not a “creative language generation” problem.

It’s a formal encoding problem with zero tolerance.

Compilers are deterministic constraint solvers for a reason.

Where ML *does* make sense

Exactly where you said:

As a decision engine inside compilation.

Not replacing:

- Instruction encoding
- Register allocation machinery
- SSA construction
- CFG construction
- ABI compliance
- Linking

But influencing:

- Inlining decisions
- Pass ordering
- Loop unrolling thresholds
- Vectorization triggers
- Prefetch insertion
- Code layout decisions

These are heuristic-heavy, cost-model-driven zones.

That's where ML can outperform static human rules.

And yes — Google already did this

Google has integrated ML into parts of LLVM for:

- Inlining heuristics
- Optimization pass ordering
- Code layout improvements (e.g., instruction placement for cache locality)

They've used reinforcement learning models to tune heuristics empirically against real workloads.

This is not speculative. It's already deployed internally.

But note the architecture:

ML decides.

Compiler enforces.

The ML model outputs a decision.

The deterministic compiler infrastructure applies it safely.

That separation is critical.

Why this works structurally

Compiler optimization is a massive search space problem.

For example:

- Inlining: exponential call graph explosion risk
- Register allocation: NP-hard in general
- Pass ordering: combinatorial explosion
- Microarchitecture modeling: effectively opaque

Human heuristics are approximations.

ML can approximate better — sometimes.

But the compiler still guarantees correctness.

That's the boundary.

Why "AI writes binary" rhetoric misses the point

Binary is not a language of abstraction.

It's a serialization format of machine instructions.

Even if AI were to "write binary," it would still need:

- Formal ISA specification constraints
- ABI rules
- Linker integration
- OS loader compatibility

In other words:

You'd just be rebuilding a compiler, but poorly.

The real shift isn't:

Skip code.

It's:

Shift heuristic layers from static cost models to learned cost models.

That's very different.

The deeper technical implication

If ML controls heuristics:

You now have:

- Potential nondeterminism (if stochastic)
- Training-data bias in performance
- Hardware-specific learned behavior
- Harder-to-reproduce builds
- Governance implications for build integrity

This is where your systems thinking becomes relevant.

You go from:

Deterministic optimization surface

→ Learned optimization surface

And that introduces:

- Stability questions
- Reproducibility questions
- Supply-chain questions

That's the real frontier.

If you want to go further, the next serious question is:

Should ML-guided optimization be:

1. Deterministically seeded?
2. Allowed to explore stochastically?
3. Benchmarked per-build and best-selected?
4. Hardware-profiled at install time?

Each of those leads to a very different compiler future.

Where do you want to take it?

You said:

demizing some binary sizes down, and it actually ended up beating humans by like 7%, which was pretty impressive. And this has been upstreamed back into LLVM back in 2021 already. But my prediction is a little bit different. My prediction is that by the end of the year, 65% odds that LLVM MLGO lists three plus ML-guided heuristics. Right now it's two. It's inlining for size and reg alloc eviction. If I had to guess, the next one would probably be loop unrolling, since it's like the next logical step. Because if you do loop unrolling improperly, you're just gonna kill the instruction cache with a ton of repeated instructions. But this maps nicely to models. You could use fixed seeds. And there's already quite a few papers talking about how it beats vanilla optimization level 3. I'll link those below.

ChatGPT said:

Your forecast is technically grounded.

Quick state check first:

LLVM's MLGO (ML-Guided Optimization) has already shipped models for:

- Inlining (size-focused)
- Register allocation eviction (RL-based advisor)

Those were upstreamed around 2021–2022 and are real production components inside LLVM.

So the trajectory isn't speculative — it's incremental expansion of advisory roles.

Why Loop Unrolling Is a Logical Next Target

Loop unrolling sits in exactly the right heuristic class:

- Strong performance sensitivity
- Direct interaction with I-cache pressure
- Impacts branch predictor behavior
- Affects register pressure
- Influences vectorization opportunities

The search surface is messy and hardware-specific.

Human heuristics use:

- Static trip-count estimation
- Code size thresholds
- Target-dependent cost tables

But real performance depends on:

- Microarchitecture
- Cache associativity
- Frontend bandwidth
- Speculation behavior

That's precisely where learned cost models outperform symbolic approximations.

So yes — loop unrolling is a plausible candidate.

The Seeded Determinism Angle

You're correct that ML guidance doesn't require nondeterminism.

There are three architectures possible:

1. Fully deterministic ML advisor

- Fixed model
- Fixed input features
- Fixed seed
- Deterministic inference

Reproducible builds preserved.

2. Stochastic exploration with fixed seed

- Exploration allowed
- Seed controls decision path
- Binary reproducible per seed

This allows multi-build benchmarking strategies.

3. Adaptive profile-guided ML

- Hardware-specific feedback
- Installation-time optimization
- Potentially non-reproducible across machines

The first two are upstream-compatible.

The third becomes controversial.

Your 65% Prediction

If the current baseline is two ML heuristics, then adding one more by year-end isn't radical.

But the gating factors aren't just technical performance.

Upstream criteria include:

- Training stability
- Inference cost
- Model size
- Cross-architecture generalization
- Build reproducibility guarantees
- Maintenance burden

LLVM culture is conservative. They won't upstream models unless:

- They are predictable
- They don't destabilize builds
- They are portable across targets

So loop unrolling would need:

- Clear wins across diverse benchmarks

- No pathological regressions
- Low inference overhead

Why This Is Bigger Than It Looks

Once you cross 3+ ML-guided heuristics:

LLVM shifts from:

Heuristic compiler with occasional ML assistance

to:

Compiler with learned policy layers embedded.

That's a structural shift.

After that point, it becomes easier to justify:

- ML pass ordering
- ML-driven vectorization decisions
- Code layout optimization expansion
- Target-specific microarchitecture tuning

The slope steepens.

The Constraint That Keeps It Grounded

The key is this:

MLGO does not replace the compiler's correctness machinery.

It replaces decision heuristics.

That boundary is why this works.

The compiler remains:

- SSA-based
- Type-safe

- ABI-compliant
- Deterministic in transformation

ML influences cost modeling.

That separation is what keeps the system sane.

Your prediction isn't hype.

It's an extrapolation of a current trajectory.

If you had to extend your prediction one layer further:

Do you think LLVM will eventually:

- Make ML default at -O3?
- Or keep it behind flags as "experimental but stable"?

Because that's where the real inflection happens.

You said:

The part is that LLVM is just such a critically important project that the maintainers are super selective about what gets accepted and upstreamed back into LLVM. Kind of the overall sentiment is only use ML solutions when they consistently beat hand-tuned heuristics with safe fallbacks. So those are my predictions for 2026. And if there's one thing that you should take away from this, it's that there is a lot of exciting stuff happening still in the field of computer science. And there's a lot of doom and gloom out there, but if you take a little bit of a deeper look, you'll see that there's still a lot of cool stuff happening that nobody seems to be talking about. As always, I will link as many relevant papers.

ChatGPT said:

That read is accurate.

LLVM is infrastructure-level critical. It underpins:

- OS toolchains

- Browsers
- Embedded firmware
- Datacenter services
- Safety-adjacent systems

So upstreaming policy is conservative by necessity.

The informal rule you described is essentially correct:

ML is acceptable only when it:

- Beats tuned heuristics consistently
- Doesn’t regress outliers badly
- Has deterministic behavior
- Has safe fallback paths
- Doesn’t explode compile time

That’s the culture.

Why LLVM Maintainers Are Cautious

The risk isn’t “ML is bad.”

The risk is:

1. Silent performance regressions on niche workloads
2. Increased build-time variance
3. Non-reproducible builds
4. Model drift across versions
5. Maintenance burden

A heuristic encoded in C++ is inspectable and diffable.

A learned policy is a trained artifact.

That changes review philosophy.

Why ML Still Wins Specific Battles

Heuristics struggle when:

- Cost models depend on opaque microarchitecture behavior

- The search space explodes combinatorially
- Human intuition saturates

Inlining and regalloc eviction fall into that class.

Loop unrolling does too.

But something like:

- Instruction encoding
- SSA construction
- Dominator tree building

Those are formal problems. ML adds no value there.

LLVM will continue slicing off heuristic-heavy decision nodes, not rewriting the compiler.

The Real Structural Shift

Once 3+ ML-guided heuristics are upstream and stable, LLVM quietly becomes:

Deterministic transformation engine

- Learned cost-policy overlays

That's subtle but important.

The compiler stops being purely rule-based and becomes partially data-trained.

Still safe.

Still deterministic.

But policy-learned.

And You're Right About the "Doom Narrative"

The public narrative around CS right now is:

- "AI replaces developers."
- "Coding is dead."
- "Everything becomes prompt engineering."

Meanwhile, the interesting work is happening in:

- Toolchain optimization
- Hardware-software co-design
- Memory model research
- Distributed systems verification
- Performance modeling
- ML-integrated compilers

It's quieter, but deeper.

If you want to sharpen your prediction further:

By 2026, do you expect:

- ML heuristics to be default-on in production builds?
- Or still opt-in under flags like `-mllvm -enable-ml-inliner`?

That's the signal to watch.

Default adoption means the trust barrier has been crossed.